

Frontend Implementation Specification for Stock Sentiment Predictor

Document Overview

This specification outlines the implementation of a pure HTML/CSS/JavaScript frontend for the Stock Sentiment Predictor system. The frontend communicates with the existing Python backend via HTTP requests to the aiohttp server (running on <http://localhost:8000>). No frontend frameworks (React, Vue, Angular) will be used—only vanilla JavaScript, HTML5, and CSS3. The interface will be responsive, modern, and provide real-time data visualization for stock predictions, sentiment analysis, and market insights.

1. Project Structure

The frontend will be organized in a clean, maintainable structure:

```
frontend/
├── index.html          # Main dashboard entry point
├── pages/
│   ├── dashboard.html  # Main dashboard view
│   ├── predictions.html # Stock predictions view
│   ├── sentiment.html  # Sentiment analysis view
│   ├── stocks.html     # Stock data browser
│   ├── history.html    # Prediction history
│   └── profile.html     # User profile and settings
├── css/
│   ├── main.css        # Global styles
│   ├── dashboard.css   # Dashboard-specific styles
│   ├── components.css  # Reusable component styles
│   └── responsive.css  # Mobile/responsive styles
├── js/
└── app.js             # Main application controller
```

```

|   ├── api.js          # API communication layer
|   ├── auth.js         # Authentication handling
|   ├── dashboard.js    # Dashboard functionality
|   ├── predictions.js  # Prediction display logic
|   ├── sentiment.js    # Sentiment analysis UI
|   ├── stocks.js       # Stock data browser
|   ├── charts.js       # Chart rendering (Chart.js)
|   ├── utils.js        # Helper functions
|   └── state.js        # Application state management
└── assets/
    ├── icons/          # SVG icons
    ├── images/         # Static images
    └── fonts/          # Custom fonts (optional)
└── lib/
    ├── chart.js        # Chart.js CDN fallback
    └── fontawesome/    # Font Awesome icons

```

2. Core Application Architecture

2.1 Single Page Application (SPA) Pattern

The frontend will implement SPA behavior without a framework using:

- **Client-side routing:** Hash-based navigation (`window.location.hash`)
- **Dynamic content loading:** Fetch and inject HTML fragments
- **State management:** Central JavaScript state object
- **Event delegation:** Centralized event handling

2.2 Application State

```

// State structure (js/state.js)
const AppState = {
  user: null,          // Current user object
  token: null,         // JWT token
  sessionId: null,     // Session ID

```

```
currentPage: 'dashboard', // Current active page
selectedSymbol: 'AAPL', // Currently selected stock symbol
predictions: [], // Cached predictions
sentimentData: [], // Cached sentiment data
stockData: [], // Cached stock data
loadingStates: { // Loading indicators
  predictions: false,
  sentiment: false,
  stocks: false
},
error: null, // Current error message
theme: 'dark' // Theme preference
};
```

2.3 API Client Layer

The API client (js/api.js) will provide a clean interface to all backend endpoints:

Authentication Endpoints:

- POST /auth/register - User registration
- POST /auth/login - User login with token retrieval
- POST /auth/logout - Session termination

Stock Endpoints:

- GET /stocks/{symbol}/data - Historical price data
- GET /stocks/{symbol}/price - Current price
- GET /stocks/{symbol}/indicators - Technical indicators
- POST /stocks/collect - Trigger data collection

Sentiment Endpoints:

- GET /sentiment/{symbol} - Sentiment data for symbol
- POST /sentiment/{symbol}/analyze - Trigger sentiment analysis

Prediction Endpoints:

- GET /predictions/{symbol} - Get predictions

- POST /predictions/{symbol}/train - Train model
- GET /predictions/{symbol}/history - Prediction history
- GET /predictions/{symbol}/backtest - Backtest results

Agent Endpoint:

- POST /agent/query - Natural language queries

Symbols Endpoint:

- GET /symbols/search - Search for symbols

3. Page Designs and Functionality

3.1 Dashboard (pages/dashboard.html)

The dashboard serves as the main landing page with key metrics and quick actions.

Components:

- **Header Navigation:** Logo, user profile dropdown, theme toggle, logout button
- **Symbol Selector:** Searchable dropdown for stock symbols with autocomplete
- **Key Metrics Cards:**
 - Current price with change percentage
 - Today's sentiment score with trend indicator
 - Next-day prediction with confidence
 - 7-day prediction range
- **Price Chart:** Interactive line chart showing 30-day price history with volume overlay
- **Sentiment Timeline:** Bar/line chart showing sentiment trends over time
- **Recent News Feed:** Latest news articles with sentiment indicators
- **Quick Actions Panel:** Buttons for common tasks (analyze sentiment, train model, get prediction)
- **AI Assistant Chat:** Collapsible chat panel for natural language queries

Interactive Elements:

- Symbol selector triggers all dashboard data refresh
- Chart tooltips show detailed values on hover

- News items expand to show full content
- AI assistant accepts text input and displays responses

3.2 Predictions Page (pages/predictions.html)

Dedicated page for detailed prediction analysis and model management.

Components:

- **Symbol Selector** with date range picker
- **Prediction Chart:**
 - Historical prices (line)
 - Predicted prices (dashed line)
 - Confidence intervals (shaded area)
- **Prediction Table:**
 - Target dates
 - Predicted prices
 - Confidence scores (progress bars)
 - Trend direction (icons)
 - Model source badges
- **Model Training Panel:**
 - Train button with progress indicator
 - Model metrics display (MSE, MAE)
 - Last training timestamp
- **Multi-Day Prediction Controls:**
 - Slider for prediction horizon (1-30 days)
 - Generate button
- **Export Options:**
 - Download predictions as CSV
 - Copy to clipboard

3.3 Sentiment Analysis Page (pages/sentiment.html)

Comprehensive sentiment visualization and analysis tools.

Components:

- **Symbol Selector** with date range
- **Sentiment Gauge:**

- Needle gauge showing current sentiment (-1 to 1)
- Color-coded (red to green)
- **Sentiment Timeline Chart:**
 - Line chart with sentiment scores
 - Annotations for significant news events
- **Sentiment Distribution:**
 - Pie/donut chart showing positive/neutral/negative ratio
 - Article count breakdown
- **News Articles Table:**
 - Title, source, date
 - Sentiment score with color coding
 - Expandable content preview
 - Link to full article
- **Word Cloud:**
 - Common terms from recent news
 - Size based on frequency
- **Analysis Controls:**
 - Analyze recent articles button
 - Date range selector

3.4 Stocks Page (pages/stocks.html)

Data browser for historical stock information and technical indicators.

Components:

- **Symbol Search:** Search with autocomplete and company info
- **Stock Overview:**
 - Current price with large font
 - Day high/low
 - 52-week range
 - Volume with average comparison
- **OHLC Chart:** Candlestick chart using lightweight-charts
- **Technical Indicators Panel:**
 - SMA (5, 20, 50 day)
 - RSI with gauge
 - MACD chart
 - Bollinger Bands

- **Data Table:**
 - Paginated historical data
 - Sortable columns
 - Export to CSV
- **Data Collection Controls:**
 - Fetch historical data button
 - Date range picker
 - Progress indicator

3.5 History Page ([pages/history.html](#))

Track prediction accuracy and review past forecasts.

Components:

- **Symbol Selector**
- **Performance Metrics Cards:**
 - MAE (Mean Absolute Error)
 - RMSE (Root Mean Square Error)
 - Prediction count
 - Accuracy trend (improving/worsening)
- **Accuracy Chart:**
 - Scatter plot of predicted vs actual
 - Perfect prediction line
 - Error distribution
- **Historical Predictions Table:**
 - Prediction date, target date
 - Predicted vs actual price
 - Error amount and percentage
 - Confidence score
 - Model source
- **Backtest Controls:**
 - Run backtest button
 - Date range selector
- **Model Performance Over Time:**
 - Line chart showing accuracy trends
 - Rolling average of errors

3.6 Profile Page (pages/profile.html)

User account management and preferences.

Components:

- **User Information:**
 - Avatar (generated from initials)
 - Username, email
 - Full name (editable)
- **Security Settings:**
 - Change password form
 - Session management (active sessions list)
 - Logout all devices button
- **Preferences:**
 - Default symbol for dashboard
 - Theme selector (light/dark/system)
 - Chart preferences (colors, indicators)
 - Notification settings (if implemented)
- **Account Stats:**
 - Total predictions made
 - Favorite symbols
 - Last login time
- **API Key Management** (if needed for external access)

4. User Authentication Flow

4.1 Login Page (login.html)

Separate page for authentication with form validation.

Components:

- **Login Form:**
 - Username/email field
 - Password field with show/hide toggle
 - Remember me checkbox

- Submit button
- **Register Link:** Navigate to registration
- **Error Display:** Show authentication errors
- **Loading State:** Disable form during submission

4.2 Registration Page (register.html)

User registration with validation.

Components:

- **Registration Form:**
 - Full name field
 - Username field (unique validation)
 - Email field (format validation)
 - Password field with strength meter
 - Confirm password field
 - Terms acceptance checkbox
- **Login Link:** Navigate to login
- **Error Display:** Show validation errors

4.3 Session Management

Token Storage:

- Store JWT token in localStorage with expiration
- Store session ID for audit purposes
- Clear storage on logout

Authentication Interceptor:

- Add Authorization header to all API requests
- Handle 401 responses (redirect to login)
- Refresh token on expiration (if implemented)

5. Charting and Visualization

5.1 Chart Library Integration

Use Chart.js for most visualizations with the following configurations:

Price Chart:

- Type: Line with gradient fill
- Dual axes: price (left) and volume (right) as bar chart
- Interactive tooltips showing all data points
- Zoom/Pan capabilities (using chart.js zoom plugin)

Candlestick Chart:

- Use lightweight-charts for OHLC data
- Overlay volume bars
- Technical indicators as separate series

Sentiment Timeline:

- Type: Line with area fill
- Color gradient from red to green based on sentiment
- Annotations for news events

Prediction Scatter:

- Type: Scatter plot
- Perfect prediction reference line
- Color-coded points by error magnitude

5.2 Chart Configuration

Define consistent styling:

- Dark theme by default with light theme option
- Responsive sizing (container-based)
- Tooltips with custom HTML formatting
- Legends with toggle series visibility

- Export as PNG option

6. Responsive Design

6.1 Breakpoints

- **Mobile:** < 768px (single column, stacked components)
- **Tablet:** 768px - 1024px (two columns, adjusted layouts)
- **Desktop:** > 1024px (full multi-column layouts)

6.2 Mobile Optimizations

- Hamburger menu for navigation
- Touch-friendly buttons (minimum 44x44px)
- Horizontal scroll for data tables
- Simplified charts with fewer data points
- Collapsible sections for detailed content

7. Real-time Features

7.1 WebSocket Integration (Optional)

If backend adds WebSocket support:

- Real-time price updates
- Live sentiment scoring
- Prediction progress indicators

7.2 Polling Mechanism

Fallback for real-time features:

- Auto-refresh every 30 seconds for price data
- Manual refresh button for other data
- Loading indicators during refreshes

8. Error Handling

8.1 User Feedback

Toast Notifications:

- Success messages (green)
- Error messages (red)
- Info messages (blue)
- Warning messages (orange)
- Auto-dismiss after 5 seconds

Loading States:

- Skeleton screens for content loading
- Spinner overlays for operations
- Disabled buttons during processing

8.2 Offline Handling

- Detect network connectivity
- Cache recent data in localStorage
- Show offline indicator
- Queue operations for when online

9. Performance Optimization

9.1 Asset Optimization

- Minify CSS and JS files
- Lazy load images and charts
- Debounce search inputs (300ms delay)
- Virtual scroll for long tables

9.2 Caching Strategy

- Cache API responses in memory for session duration
- Implement request deduplication
- Use browser cache for static assets

10. Accessibility (A11y)

10.1 ARIA Labels

- All interactive elements have descriptive labels
- Chart data includes aria descriptions
- Form fields have associated labels

10.2 Keyboard Navigation

- Tab order follows logical flow
- Focus indicators visible
- Skip to main content link
- Shortcut keys for common actions

10.3 Color Contrast

- WCAG AA compliance for text contrast
- Color isn't the only indicator (use icons/text)
- High contrast mode support

11. Browser Support

- Chrome (latest 2 versions)
- Firefox (latest 2 versions)
- Safari (latest 2 versions)
- Edge (latest 2 versions)
- Mobile browsers (iOS Safari, Chrome Mobile)

12. Implementation Priority

Phase 1: Core Structure (Week 1)

- Project scaffolding and file structure
- Basic HTML/CSS framework
- API client implementation
- Authentication flow
- Dashboard skeleton

Phase 2: Main Features (Week 2)

- Dashboard with price and sentiment charts
- Predictions page with basic functionality
- Stock data browser
- User profile

Phase 3: Advanced Features (Week 3)

- Sentiment analysis page with visualizations
- History and backtesting
- AI assistant integration
- Real-time updates

Phase 4: Polish (Week 4)

- Responsive design refinement
- Error handling and loading states
- Performance optimization
- Cross-browser testing
- Accessibility improvements

13. Testing Requirements

13.1 Manual Testing

- All API endpoints integration
- Form validation and submission
- Responsive layout on all devices
- Cross-browser compatibility
- Error scenarios (network failure, invalid data)

13.2 User Acceptance Testing

- Navigation flow intuitive
- Data displays correctly
- Charts are readable and interactive
- Mobile experience satisfactory

14. Deployment

14.1 Static File Serving

- Frontend files served by a simple HTTP server (nginx, Apache, or Python's `http.server`)
- No build step required (vanilla HTML/CSS/JS)
- CORS configured for backend API

14.2 Environment Configuration

- Backend API URL configurable (default: <http://localhost:8000>)
- Theme preference persisted in `localStorage`
- Session timeout configuration

15. Additional Considerations

15.1 SEO (if needed)

- Meta tags for social sharing
- Semantic HTML structure
- Sitemap generation (if public)

15.2 Analytics

- Page view tracking (optional)
- Error tracking integration
- Performance monitoring

15.3 Security

- XSS prevention (sanitize user input)
- CSRF protection (using JWT token)
- Secure storage of tokens (HttpOnly cookies if possible)

This specification provides a complete blueprint for implementing the frontend with pure web technologies. The implementation should focus on creating a responsive, user-friendly interface that effectively communicates the powerful capabilities of the sentiment analysis and prediction backend. All components should be built with vanilla JavaScript, HTML, and CSS, maintaining simplicity while delivering professional-grade functionality.